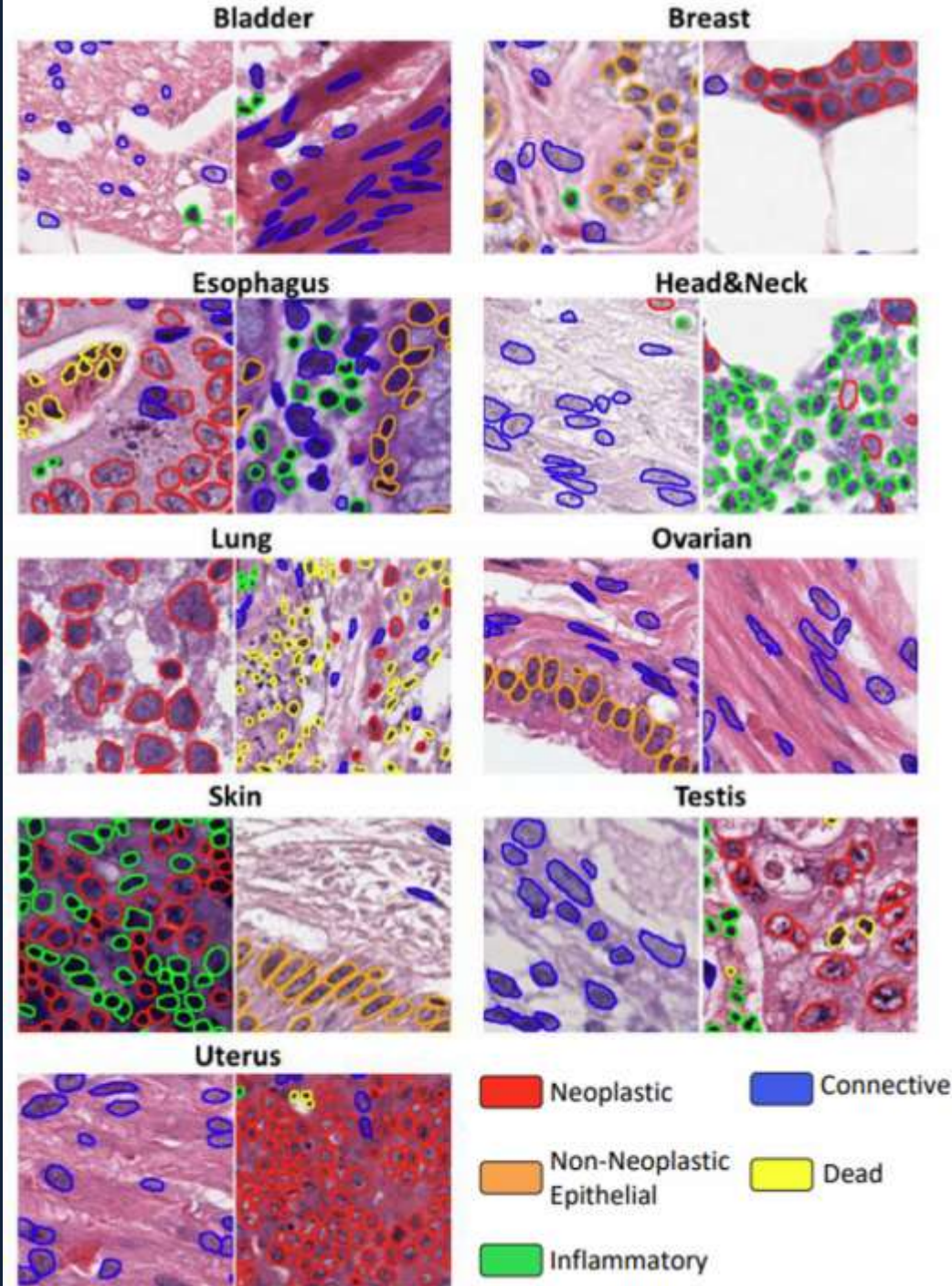


Nuclei instance segmentation across diverse tissues

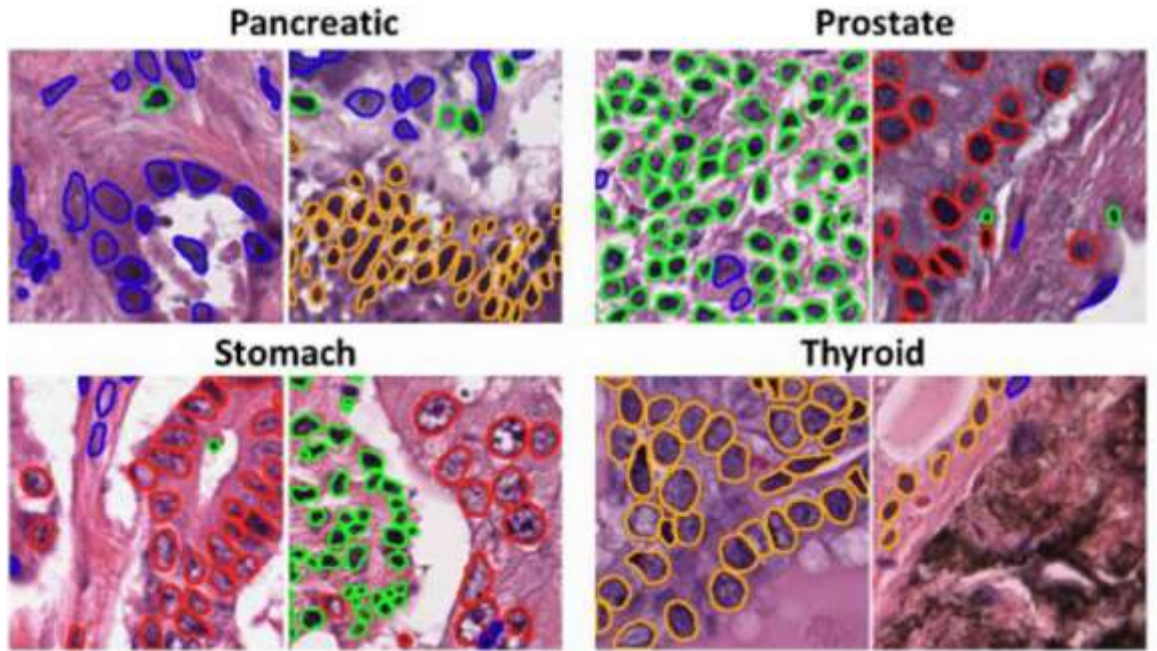
PanNuke and HoVer-Net

Project E presentation

Team members: Zhiyu Song, Yuchen Ai, Xinhao Jin, Zheng Jian



Touching nuclei must remain separate objects



Input

A 256×256 H&E pathology patch

Target output

One mask and one class label for every nucleus

Why instance level matters

A merged pair changes cell counts, boundaries, spatial relationships, and downstream tissue measurements.

Main question: can one model separate and classify nuclei consistently across diverse tissues?

PanNuke provides diverse tissue and instance labels

7,904

H&E patches

19

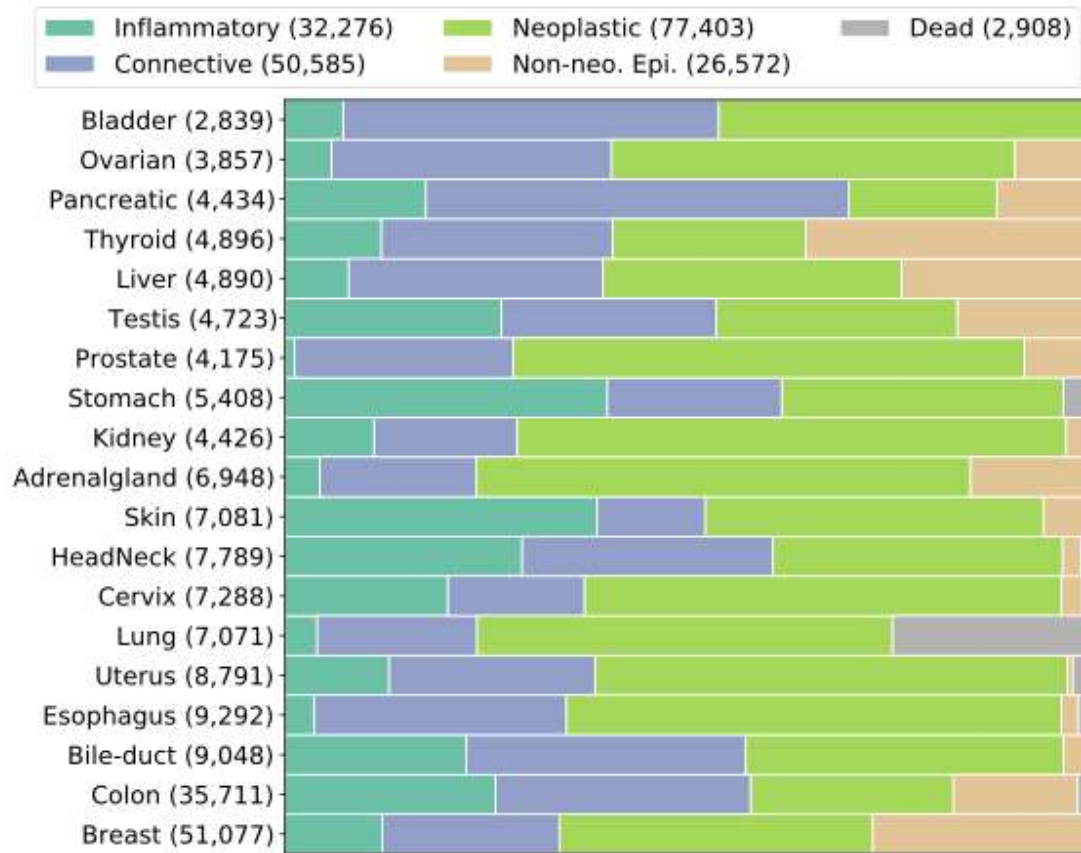
tissue types

189,744

verified nuclei

5

nucleus classes



Five positive classes

Neoplastic

Inflammatory

Connective

Dead

Non-neoplastic epithelial

Where to obtain it

Official project page jgamper.github.io/PanNukeDataset/

Programmatic option `PathML PanNukeDataModule(download=True)`

Record the exact release and checksums because public sources report both about 190k and about 205k labels.

Published HoVer-Net results already vary by tissue

PanNuke benchmark

The dataset paper compared DIST, Mask R-CNN, Micro-Net, and HoVer-Net on three official folds.

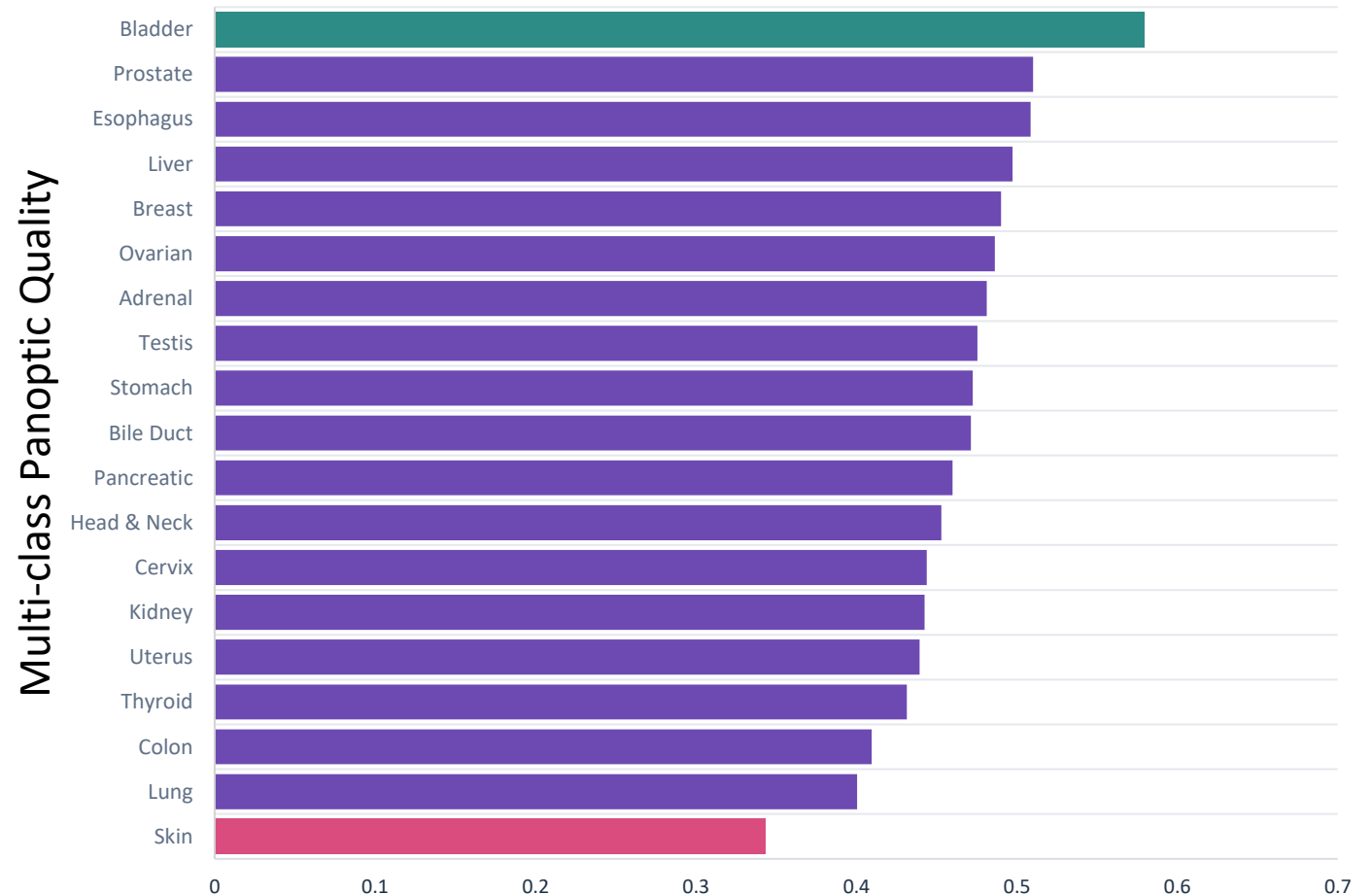
Chosen baseline

HoVer-Net achieved the strongest average results and has official code plus PanNuke checkpoints.

Why our project differs

We will reproduce one model, then quantify its worst tissue and class failures before testing one controlled change.

Published HoVer-Net mPQ by tissue (higher is better)



One model, three immediate investigations

Can HoVer-Net perform consistently across tissue types and nucleus classes?

1

Dataset audit

Count patches and nuclei by tissue, class, and fold.
Inspect random and difficult overlays.

2

Baseline reproduction

Run the official PanNuke checkpoint,
then train one fold with fixed settings.

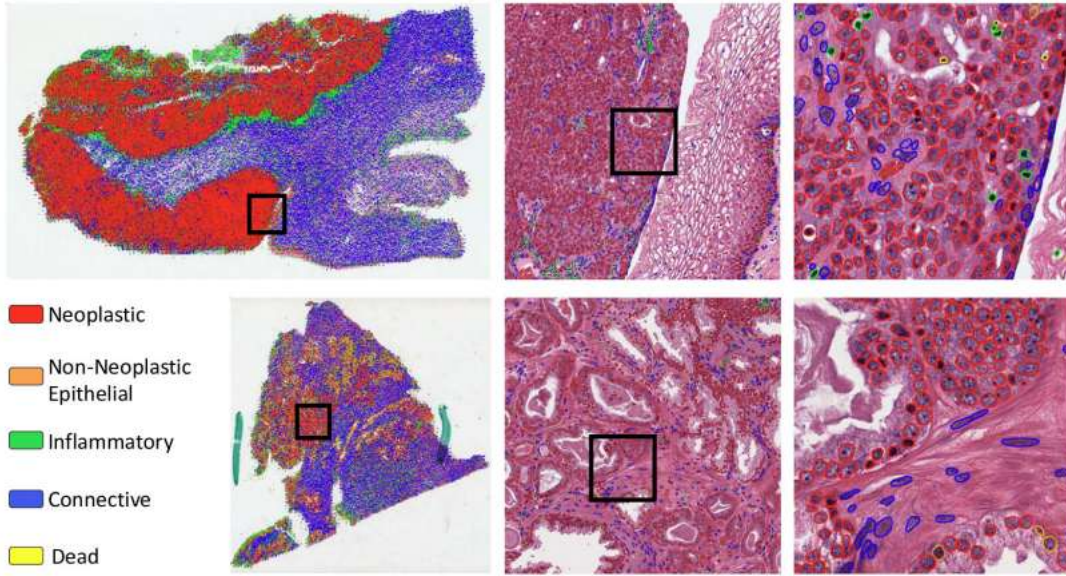
3

Failure map

Measure tissue-by-class PQ and classify merged,
split, missed, and mistyped nuclei.

**Improvement choice follows the error evidence.
The current candidate is class-balanced NC loss for rare nuclei.**

Exploration first, then freeze the official folds



EDA figures for the proposal

- 1 Counts by tissue, class, and fold
- 2 Image and mask overlays with easy and hard examples
- 3 Object size, crowding, stain intensity, and empty or corrupted patches

Split decision

**Use PanNuke's three published folds for comparison.
Save all patch IDs and do not tune on the held-out test fold.**

Limitation: patient-level identifiers are not part of the standard patch release, so the official split is the reproducible default.

Instance metrics reveal merges, splits, and missed nuclei

			Error taxonomy	
mPQ	Primary score	Equal-weight class PQ, reported by tissue and class	Merged	several true nuclei become one prediction
bPQ	Object quality	Class-agnostic instance separation	Split	one true nucleus becomes several predictions
F1	Detection	Precision and recall for matched nuclei	Missed	no prediction matches the true nucleus
AJI	Complement	Overlap with penalties for unmatched instances	Mistyped	instance found, class label is wrong

Required report

**Overall result + tissue table + class table +
tissue-by-class heat map + representative error gallery**

Main comparison: standard HoVer-Net versus one targeted modification on the same held-out patches.

A staged run plan keeps the project reproducible

Checkpoint	Official PanNuke weights	Validate preprocessing, post-processing, and metrics
Small run	Subset or one fold	Confirm losses, output maps, and memory use
Full baseline	Three official rotations	Store predictions for subgroup analysis
Ablation	One targeted change	Repeat the same split and evaluation code

Compute

Laptop

EDA and unit tests

Colab

Checkpoint and short run

GPU cluster

Full folds and repeats

Resource estimate

One GPU with 8–16 GB VRAM for HoVer-Net.

The cluster is preferred for three folds and repeated comparisons.

Save split IDs, Git commit, environment, seed, config, logs, checkpoints, and the exact metric script.

Four weeks from data access to a testable plan

Week 1
9/22

Access data and read key papers

Inventory files, labels, and release

Week 2
9/29

Create EDA figures and freeze metrics

Run official checkpoint inference

Week 3
10/6

Train on a subset or one fold

Validate outputs and metric code

Week 4
10/13

Baseline error analysis

Draft and finalize the proposal

Proposal due Thursday 10/15

Minimum
checkpoint inference +
verified PQ pipeline

Target
one-fold baseline +
subgroup errors

Stretch
full folds + improvement
pilot

Backup plan: if the original training environment blocks progress, evaluate the official checkpoint and document the limitation.

Open questions for the project plan

**Thank you for listening.
We welcome feedback on scope, evaluation, and
compute feasibility.**